

Image Classification using AWS SageMaker

I used AWS Sagemaker to train a pretrained model that can perform image classification by using Sagemaker profiling, debugger, and hyperparameter tuning. I will be using the dog breed dataset to train a model to correctly identify the dog breed of a dog in a picture.

Project Set Up and Installation

Entered AWS through the gateway in the course and opened SageMaker Studio and downloaded the starter files to my Sagemaker directory.

Dataset

The dataset is the dogbreed classification dataset. The image classifier can take in any type of dataset for training, however the dog breed dataset is readily available and provides many images for training. This dataset was provided by Udacity and is available through AWS.

Access

The data was downloaded and unzipped to my sagemaker directory. From there the unzipped package was uploaded to my S3 bucket through the AWS Gateway.

Hyperparameter Tuning

RESNET50 was the model chosen for the image classifier due to it providing more accuracy than RESNET18. The hyperparameters chosen for tuning were.

1. Batch size
2. learning rate
3. epochs

Learning rate is how quickly the weights are adjusted, so if the weights are adjusted too quickly then the model may overshoot in either direction in getting to the optimal set of weights. The learning rate chosen was between 0.001 and 0.1.

Batch size is the number of samples used to make an evaluation. After each batch the weights are adjusted, so if the batch size is very large training will be quick but the model will not be accurate, while small batch sizes will increase training time and the model will be more accurate. Batch sizes chosen were 32, 64, 128 and 256, anything bigger would need more memory and possibly more processors.

Epochs is the number times to retrain the model to make adjustments to the weights. While retraining helps to make a better model, too much training will cause overfitting of the data. Epochs chosen were 1 through 4,

- Completed training jobs

Training jobs Info					
Search training jobs					
	Name	Creation time	Duration	Job status	
☐	pytorch-training-2024-08-17-03-16-34-317	8/16/2024, 8:16:35 PM	-	InProgress	
☐	pytorch-training-2024-08-17-03-04-52-134	8/16/2024, 8:04:53 PM	a minute	Stopped	
☐	pytorch-training-2024-08-17-01-09-33-139	8/16/2024, 6:09:34 PM	an hour	Completed	
☐	pytorch-training-2024-08-16-22-44-45-888	8/16/2024, 3:44:46 PM	an hour	Completed	
☐	pytorch-training-240816-2050-004-90cfbb39	8/16/2024, 2:55:25 PM	35 minutes	Completed	
☐	pytorch-training-240816-2050-003-724189d2	8/16/2024, 2:25:46 PM	an hour	Completed	
☐	pytorch-training-240816-2050-002-37940642	8/16/2024, 1:50:25 PM	an hour	Completed	
☐	pytorch-training-240816-2050-001-6dc1e5c9	8/16/2024, 1:50:23 PM	32 minutes	Completed	

- Logs metrics during the training process

Monitor

Access logs for debugging and progress reporting. [Learn more](#)

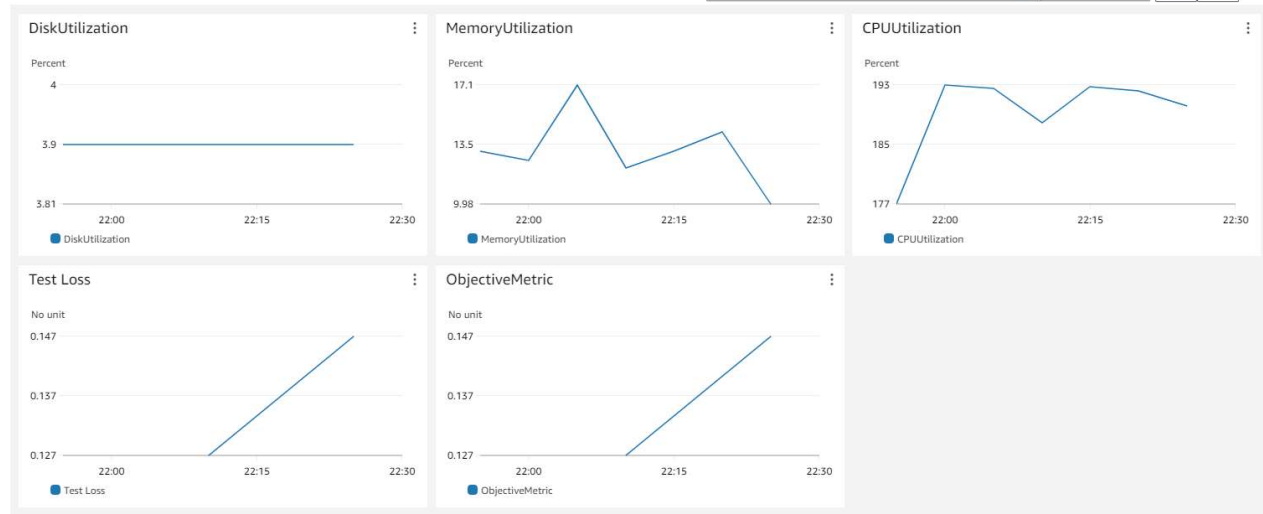
[View algorithm metrics](#)

[View instance metrics](#)

[View logs](#)

[Search logs](#)

2024-08-16T21:55:30 > 2024-08-16T22:30:01 UTC timezone



- Three hyperparameters tuned

Hyperparameter Tuning

TODO: This is the part where you will finetune a pretrained model with hyperparameter tuning. Remember that you have to tune a minimum of two hyperparameters. However you are encouraged to tune more. You are also encouraged to explain why you chose to tune those particular hyperparameters and the ranges.

Note: You will need to use the `hpo.py` script to perform hyperparameter tuning.

```
#TODO: Declare your HP ranges, metrics etc.
hyperparameter_ranges = {
    "lr": ContinuousParameter(0.001, 0.1),
    "epochs": IntegerParameter(1, 4),
    "batch-size": CategoricalParameter([32, 64, 128, 256]),
}
```

- Best hyperparameters from all the training jobs

Best training job summary This training job is the best training job for only this hyperparameter tuning job.			
Name pytorch-training-240816-2050-002-37940642	Status ✔ Completed	Objective metric Test Loss	Value 0.026200000196695328

Best training job hyperparameters 			
Name	Type	Value	
_tuning_objective_metric	FreeText	Test Loss	
batch-size	Categorical	"32"	
epochs	Integer	4	
lr	Continuous	0.013351091326965773	
sagemaker_container_log_level	FreeText	20	
sagemaker_estimator_class_name	FreeText	"PyTorch"	
sagemaker_estimator_module	FreeText	"sagemaker.pytorch.estimator"	
sagemaker_job_name	FreeText	"pytorch-training-2024-08-16-20-50-17-672"	
sagemaker_program	FreeText	"hpo.py"	
sagemaker_region	FreeText	"us-east-1"	
sagemaker_submit_directory	FreeText	"s3://sagemaker-us-east-1-065326103865/pytorch-training-2024-08-16-20-50-17-672/source/sourcedir.tar.gz"	

Debugging and Profiling

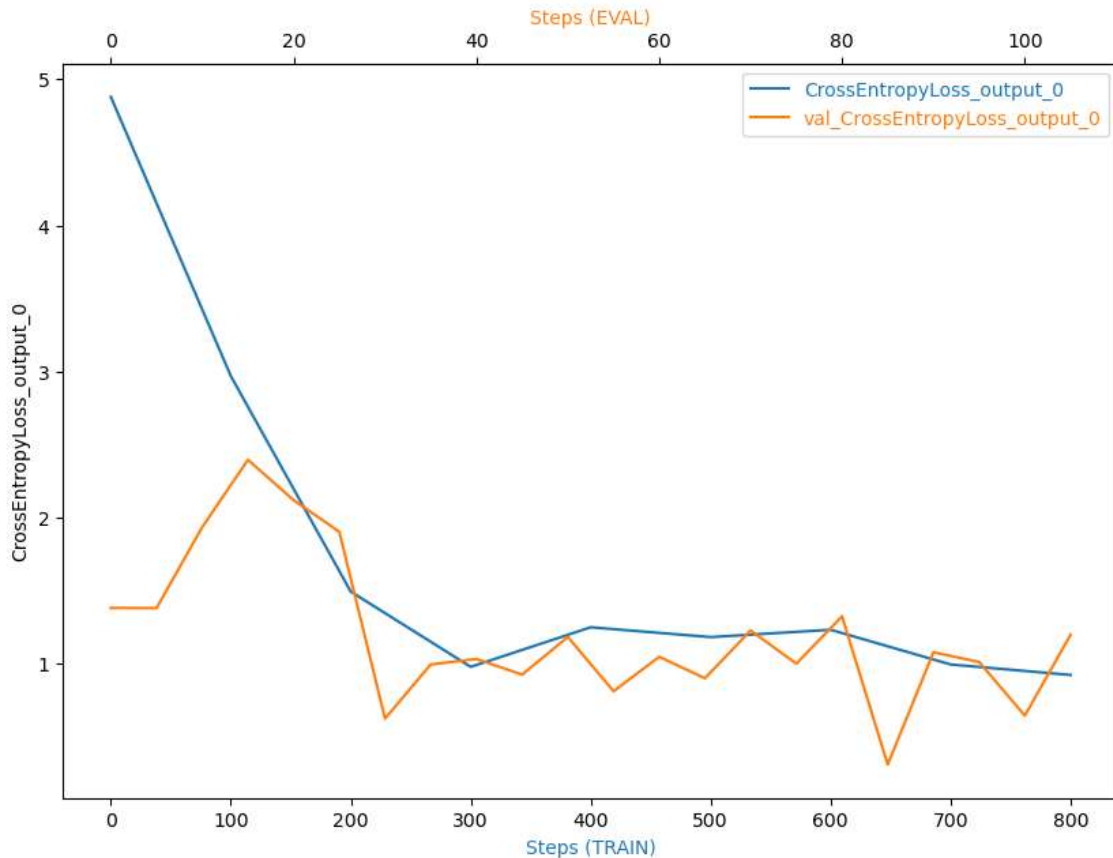
Debugger rules chosen were.

- 1. Vanishing Gradient
- 2. Overfit
- 3. Overtraining
- 4. Poor weight initialization
- 5. Loss not decreasing

Profiler rules chosen were

- 1. LowGPU utilization
- 2. ProfilerReport

From the Debugger Plot it appears that the loss function is decreasing for both the training and eval cycles.



Results

From the profiler report, it shows that no rules were triggered. However the rules with the most datapoints processed were:

1. Batchsize
2. CPUBottleneck
3. IOBottleneck

These rules with the most datapoints are probably due to the CPU being utilized all the time, I never checked to see if a GPU was available to use. Most of the time GPU's are not available.

Model Deployment

To deploy the trained model another pytorch model was created using the model data/artifact from the trained model with the best hyperparameters and a new script that contains the functions for the model, input and prediction/output.

1. `model_fn` - loads the saved model from the training and the model configuration.
2. `input_fn` - takes in the input passed in to the function that invokes the endpoint
3. `predict_fn` - transforms the input the same way the training data was transformed, performs inference and returns the result

- Endpoint Summary

Amazon SageMaker > Endpoints > pytorch-inference-2024-08-18-22-47-11-911

pytorch-inference-2024-08-18-22-47-11-911

Endpoint summary		
Name pytorch-inference-2024-08-18-22-47-11-911	Status  InService	Type Real-time
ARN arn:aws:sagemaker:us-east-1:065326103865:endpoint/pytorch-inference-2024-08-18-22-47-11-911	Creation time Sun Aug 18 2024 15:47:12 GMT-0700 (Pacific Daylight Time)	Last updated Sun Aug 18 2024 15:50:44 GMT-0700 (Pacific Daylight Time)
URL https://runtime.sagemaker.us-east-1.amazonaws.com/endpoints/pytorch-inference-2024-08-18-22-47-11-911/invocations Learn more about the API 	Model container logs /aws/sagemaker/endpoints/pytorch-inference-2024-08-18-22-47-11-911	Alarms 0 alarms

- Invoking the Endpoint

```
# TODO: Run an prediction on the endpoint

from sagemaker.predictor import Predictor
from sagemaker.serializers import IdentitySerializer

predictor.serializer = IdentitySerializer('image/jpeg')

pick = generate_test_case()
s3.download_file(bucket, pick, pick.split('/')[4])

with open(pick.split('/')[4], "rb") as file:
    body = file.read()
    response = predictor.predict(body)
```

Standout Suggestions

Would have tried to create my own dockerfile to use to run inferences on sample images, but was running out of budget.